

# Java vs. JavaScript: Key Differences

Personal Programming Notes & Reference Sheet | GitHub Edition

## 1. Variable Declaration & Type Systems

**Java** is *statically typed*. You must explicitly declare the exact data type before using a variable, and that type can never change.

```
int myNumber = 5;
String myText = "Hello";
char myLetter = 'a';
float myDecimal = 5.99f;
boolean myBool = true;
```

**JavaScript** is *dynamically typed*. You do not declare specific types. You use universal keywords (let, const, var), and variables can freely change types at runtime.

```
let myVar = 5; // Currently a number
myVar = "Hello"; // Now it's a string!
```

## 2. The Single Quote (') Rule

**Java:** Single quotes ' ' are strictly reserved for the char primitive type (a single 16-bit Unicode character). Strings *must* use double quotes " ".

```
char standardChar = 'a'; // Valid
char errorChar = "a"; // Compiler Error!
String word = "hello"; // Valid
```

**JavaScript:** There is no separate char type. Single quotes ' ' and double quotes " " are exactly identical and both represent regular string types.

```
let text1 = 'a';
let text2 = "a"; // Both are regular strings!
```

## 3. String Concatenation & Chaining Rule

Both languages follow standard **operator precedence** and **left-to-right evaluation**. However, how they convert types along the way differs dramatically because of Java's strict numeric promotion vs. JavaScript's aggressive type coercion.

| Expression | Java Behavior & Result                                  | JavaScript Behavior & Result                                  |
|------------|---------------------------------------------------------|---------------------------------------------------------------|
| 'a' + 'b'  | Adds ASCII codes (97 + 98).<br><b>Result: 195</b> (int) | Concatenates regular strings.<br><b>Result: "ab"</b> (string) |

| Expression              | Java Behavior & Result                                                                                                                                         | JavaScript Behavior & Result                                                                                     |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| "The sum is " + (x + y) | <b>Identical Behavior:</b> Parentheses () resolve first, forcing mathematical addition. The result is then glued to the text.<br><b>Result:</b> "The sum is 7" | <b>Identical Behavior:</b> Matches Java precisely. Resolves math inside parentheses before string concatenation. |
| "Result: " + x + y      | <b>Identical Behavior:</b> Evaluates left-to-right. String at front forces the rest of the chain into string concatenation.<br><b>Result:</b> "Result: 52"     | <b>Identical Behavior:</b> String at front acts like a domino, forcing everything to the right into text.        |

**The Golden Rule of Concatenation:** If a string is involved in a chain of + operators, the statement executes from left to right. Once a string is introduced, everything to its right gets converted into text—unless wrapped in parentheses () or evaluated by higher-priority math operators like multiplication \*. The final result of the entire statement will always be a single **String**.